

interview question on spring in java i just for spring topic
other topic i will take one by one

Perfect 👍 Let's focus **only on Spring Framework** (not Spring Boot, not Spring MVC — just core Spring concepts).

Here's a categorized list of **commonly asked Spring interview questions** with clear, short explanations you can memorize quickly 📌

Core Spring Framework Interview Questions

◆ 1. What is the Spring Framework?

Spring is a **lightweight, open-source framework** for building enterprise-level Java applications.

It provides **inversion of control (IoC)** and **dependency injection (DI)** to make code loosely coupled and easier to test.

◆ 2. What are the main features of Spring?

- **IoC (Inversion of Control)**
- **Dependency Injection (DI)**
- **Aspect-Oriented Programming (AOP)**
- **Transaction Management**
- **Modular architecture**
- **Integration with frameworks (JPA, Hibernate, JMS, etc.)**

◆ 3. What is Inversion of Control (IoC)?

IoC means **the control of creating and managing objects is given to the Spring container** instead of the developer.

Example: Instead of using `new` keyword, Spring creates objects (beans) and injects dependencies.

◆ 4. What is Dependency Injection (DI)?

DI is a design pattern where an object's **dependencies are injected by a container** (like Spring) rather than being created inside the object.

Types of DI:

- **Constructor Injection**
- **Setter Injection**
- **Field Injection (via @Autowired)**

◆ 5. What is a Spring Bean?

A **bean** is any Java object that is **managed by the Spring container**.

Defined in XML, Java Config, or via annotations (`@Component` , `@Service` , etc.)

◆ 6. What is the Spring Container?

The **container** is responsible for managing the lifecycle of beans — creation, wiring, configuration, and destruction.

Types of containers:

- `BeanFactory` (basic container)
- `ApplicationContext` (advanced container with extra features)

◆ 7. Difference between BeanFactory and ApplicationContext?

Feature	BeanFactory	ApplicationContext
Loading	Lazy	Eager
AOP, Event Handling	Not supported	Supported
Internationalization	No	Yes

Feature	BeanFactory	ApplicationContext
Best For	Lightweight apps	Enterprise apps

◆ 8. What are the different scopes of a Spring Bean?

- `singleton` → One instance per Spring container (default)
- `prototype` → New instance for every request
- `request` → One per HTTP request (web only)
- `session` → One per HTTP session
- `application` → One per web app lifecycle

◆ 9. What is `@Component`, `@Service`, `@Repository`, and `@Controller`?

All are **stereotype annotations** used for auto-detection by Spring.

Annotation	Usage
<code>@Component</code>	Generic Spring-managed bean
<code>@Service</code>	Business logic layer
<code>@Repository</code>	DAO layer (handles database exceptions)
<code>@Controller</code>	Web layer (MVC controller)

◆ 10. How does Spring perform Dependency Injection?

Spring performs DI in 3 ways:

1. **XML Configuration**
2. **Annotation-based** (`@Autowired` , `@Qualifier` , `@Value`)
3. **Java-based Configuration** (`@Configuration` , `@Bean`)

◆ 11. What is `@Autowired` used for?

@Autowired tells Spring to **automatically inject the dependency** by type.

If multiple beans of the same type exist, use @Qualifier("beanName") to specify which one.

◆ 12. What is @Configuration and @Bean?

- @Configuration marks a class that defines bean methods.
- @Bean tells Spring that the method returns an object that should be registered as a bean.

java

 Copy code

```
@Configuration
public class AppConfig {
    @Bean
    public UserService userService() {
        return new UserService();
    }
}
```

◆ 13. What is Spring AOP?

AOP (Aspect-Oriented Programming) separates **cross-cutting concerns** (like logging, transactions, security) from business logic.

Key Concepts:

- **Aspect** – module of cross-cutting logic
 - **Advice** – action taken at a join point (before, after, around)
 - **Join Point** – a point during program execution (e.g., method call)
 - **Pointcut** – expression that defines where advice applies
-

◆ 14. What is a proxy in Spring AOP?

Spring uses **proxies** (JDK dynamic proxies or CGLIB) to apply AOP advice around target objects without modifying the code.

◆ 15. What is the lifecycle of a Spring Bean?

1. Instantiation
2. Dependency Injection
3. `setBeanName()` (optional)
4. `setBeanFactory()` (optional)
5. `@PostConstruct` (init method)
6. Bean ready to use
7. `@PreDestroy` (destroy method)

◆ 16. What is the use of `@PostConstruct` and `@PreDestroy`?

They mark methods to run **after** bean initialization and **before** destruction respectively.

◆ 17. What is the difference between `@ComponentScan` and `@Import`?

- `@ComponentScan` → Automatically scans packages for beans
- `@Import` → Manually imports other configuration classes

◆ 18. How can we make a bean lazy initialized?

Use:

```
java
```

 Copy code

```
@Lazy
@Component
public class MyBean { ... }
```

This tells Spring to create the bean only when it is first requested.

◆ 19. What are some advantages of using Spring Framework?

- Loose coupling
- Easy testing

- Declarative transaction management
 - Integration with other frameworks
 - Simplified enterprise application development
-

◆ **20. What is the difference between tight coupling and loose coupling?**

- **Tight coupling:** One class directly creates and depends on another.
 - **Loose coupling:** Dependencies are injected externally (e.g., via Spring DI), making code flexible and testable.
-

Would you like me to cover **Spring Boot interview questions next** (as the next topic after this)?

It's the most frequently asked after core Spring.



Is this conversation helpful so far?

